

PARIVARTANA: A Three-Stage Neural Transpiler for COBOL-to-Python Modernization

MS453 Artificial Intelligence and Natural Language Processing

Aarthy Ramachandran
Northwestern University

Abstract

Roughly 800 billion lines of COBOL still run the world’s banks, insurers, and government systems, yet the workforce able to maintain that code is shrinking. PARIVARTANA (Sanskrit for “transformation”) is a three-stage natural-language-processing system that converts legacy COBOL programs into idiomatic Python 3. The system combines a deterministic, parser-free preprocessor; a Stage-2 translator that pairs a rule-based template engine with a curriculum-trained, LoRA-fine-tuned CodeT5+ neural model; and a Stage-3 large-language-model refinement layer guarded by a DeBERTa natural-language-inference semantic validator. This report recapitulates the project from research question through data engineering, architecture, and results. Following advisor feedback, the original ANTLR4 grammar parser was removed in favour of a thin preprocessor that ingests the entire NIST COBOL Test Suite with zero Stage-1 failures, and the scope was concentrated on the NIST and IBM corpora and a curriculum-trained CodeT5+. On a full NIST evaluation judged by GPT-4o-mini, the rule-based draft reaches a 0.69 weighted correctness score while the neural model reaches 0.54; a controlled 50-program comparison under a local judge preserves this ordering. The central empirical finding is that execution-verified COBOL-to-Python training pairs are extremely scarce, which currently bounds neural quality. We document the methods, the engineering challenges and their solutions, and a concrete path to close the remaining gap.

1. Introduction: Project, Challenges, and Data

1.1 Research Question and Motivation

Hand-rewriting COBOL costs an estimated one to five dollars per line, making full modernization a multi-billion-dollar problem (Micro Focus 2022). The research question for this project is whether a hybrid pipeline—combining deterministic parsing, transfer-learned neural translation, and large-language-model refinement—can produce Python that is not merely runnable but readable and semantically faithful, despite the near-total absence of COBOL-to-Python training data. Two well-documented failure modes motivate the design: rule-based converters emit Python that runs but reads like transliterated COBOL, while neural code models have seen almost no COBOL during pre-training (COBOL is under 0.01% of typical code corpora).

1.2 Challenges

Three challenges dominated the work. First, COBOL is column-sensitive 1960s fixed-format code: columns 1–6 hold line numbers, column 7 is a marker, and columns 8–72 hold the statement, so naïve tokenization fails. Second, the supervised signal is missing—there is no large parallel COBOL-to-Python corpus, so training targets must be manufactured and verified. Third, evaluation is hard: most NIST programs are conformance-harness programs that exit non-zero when run standalone, so execution-based scoring only applies to a small self-contained subset, and a learned judge must carry the rest.

1.3 Data Sources

After the proposal scope cut, four datasets are active (tagged P0); the remaining datasets were deferred.

Table 1 summarizes the active sources and their role in the pipeline.

Dataset	Role	Use in pipeline
NIST COBOL Test Suite (~2,400 programs)	TEST	Gold evaluation corpus; 459 complete programs scored after excluding copybook fragments.
IBM Open COBOL Samples (~130 files)	TRAIN	Real enterprise code (banking, insurance, HR); primary source of runnable programs.
CodeXGLUE code-to-code (10,300 pairs)	TRAIN	Transfer corpus. Note: the available subset is Java→C#, not Java→Python (see §4).
FEVER NLI Corpus (185k claims)	TRAIN	Trains the DeBERTa semantic validator that flags meaning drift.
COBOL Identifier Dictionary (12.5k)	SUPPORT	Maps cryptic COBOL names to readable Python names during refinement.

Table 1. Active datasets after the proposal scope cut.

2. From Proposal to Implementation: How the Pipeline Evolved

The project changed substantially between the proposal and the working system, largely in response to advisor feedback that recommended dropping the grammar parser and concentrating effort on the neural translation and refinement. Table 2 contrasts the initial design with the current implementation.

Element	Initial proposal	Current pipeline
Stage 1 parsing	Full ANTLR4 COBOL-85 grammar producing a complete AST.	Thin parser-free preprocessor + recursive-descent over a ~25-verb subset; 0 Stage-1 failures on full NIST.
Datasets	Nine datasets (~285k–462k samples).	Four active (NIST, IBM, CodeXGLUE, FEVER) + identifier dictionary.

Stage 2	CodeT5+ transfer + curriculum (planned).	Rule-based draft + curriculum-trained CodeT5+ fine-tuned with LoRA (trained checkpoint exists).
Stage 3	GPT-4 refinement.	Provider-agnostic LLM refinement + DeBERTa NLI validator with lexical fallback.
Training data	Assumed available.	Manufactured via execution-verified LLM distillation (gold) plus tagged silver.
Citations	Mixed style.	Chicago author-date (per advisor CMOS guidance).

Table 2. Proposal versus implemented pipeline.

The single most consequential change was eliminating the ANTLR4 grammar. An ablation across the full NIST suite confirmed the advisor’s recommendation: the parser-free preprocessor ingested every one of the 500-plus files with zero Stage-1 failures, freeing the time budget for the machine-learning work the proposal identified as its core contribution.

3. NLP Architecture and Methods

The system is a three-stage pipeline (Figure 1). Each stage is independently testable, and Stage 2 can run in either rule-based or neural mode so the two can be compared on identical inputs.

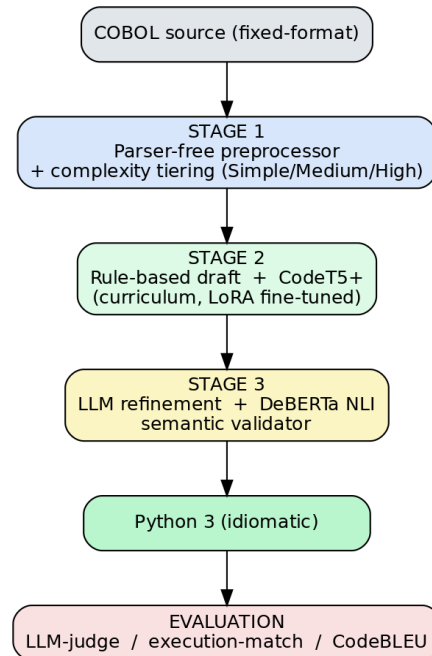


Figure 1. End-to-end PARIVARTANA pipeline, from COBOL source through the three stages to evaluation.

3.1 Stage 1: Parser-Free Preprocessor and Complexity Tiering

Stage 1 strips fixed-format column metadata, normalizes EBCDIC to UTF-8, lexes a ~25-verb COBOL subset, and builds a lightweight abstract syntax tree. A complexity score derived from tree depth, unique verbs, cross-references, and difficult constructs sorts each program into Simple, Medium, or High tiers. These tiers drive the curriculum in Stage 2 and the human-review flag in Stage 3.

3.2 Stage 2: Rule-Based Draft and Curriculum CodeT5+

Stage 2 produces the translation. The rule-based engine is a fast, deterministic baseline that maps AST nodes to Python templates. The neural translator is CodeT5+ (220M parameters; Wang et al. 2023), an encoder-decoder transformer (Vaswani et al. 2017). Figure 3 shows its architecture, recovered directly from the trained checkpoint: shared embeddings over a 32,105-token vocabulary, a twelve-block encoder, a twelve-block decoder with cross-attention, and a language-model head, totalling roughly 223 million parameters.

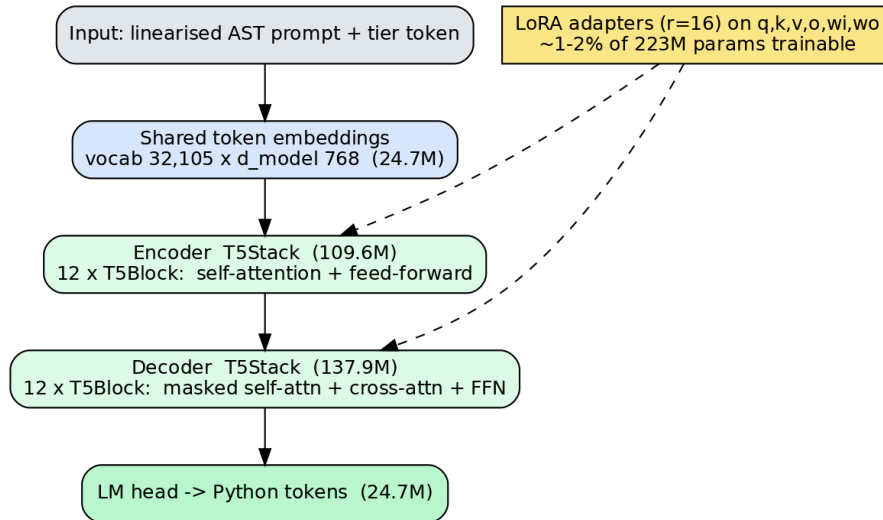


Figure 2. CodeT5+ encoder-decoder architecture with parameter counts, and the LoRA adaptation used for fine-tuning.

To make fine-tuning tractable on an Apple M3 Pro, we used Low-Rank Adaptation (LoRA; Hu et al. 2021) with rank 16 on the attention and feed-forward projections, training only one to two percent of the parameters. Training follows a competence-based curriculum (Bengio et al. 2009; Platanios et al. 2019; Kühne et al. 2025):

examples are released from Simple to Medium to High as validation performance plateaus. Figure 4 depicts the two-dimensional curriculum, which orders examples by both language similarity and AST complexity, and which separates execution-verified “gold” pairs from unverified “silver” pairs.

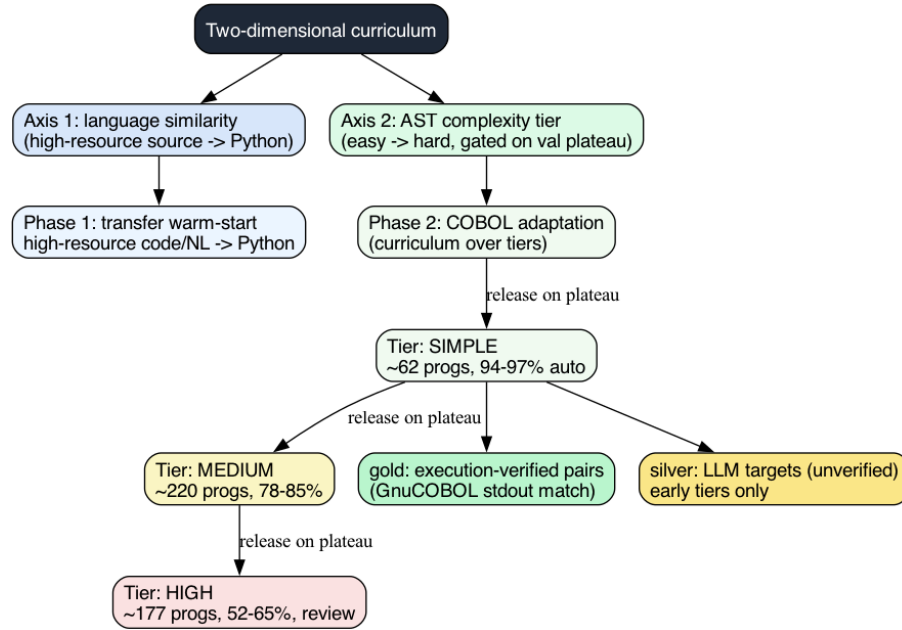


Figure 3. Two-dimensional curriculum: language-similarity warm-start crossed with Simple/Medium/High AST-complexity tiers, with gold/silver provenance.

3.3 Stage 3: LLM Refinement and NLI Semantic Validation

Stage 3 sends the Stage-2 draft to a large language model that renames variables using the identifier dictionary, writes docstrings, and rewrites the code to be idiomatic. A DeBERTa model fine-tuned on the FEVER corpus (Thorne et al. 2018) then checks whether the COBOL comment and the generated Python docstring still entail the same meaning, flagging “silent” errors that run correctly but compute the wrong thing. A lexical fallback runs when the model is unavailable.

3.4 Knowledge Representation: the AST as a Knowledge Graph

The Stage-1 AST functions as a typed knowledge graph of the program—Program nodes link to Divisions, which link to paragraphs and statements—and is linearized into the neural model’s prompt together with a tier token (Figure 5). This representation lets the model condition on program structure rather than raw character columns.

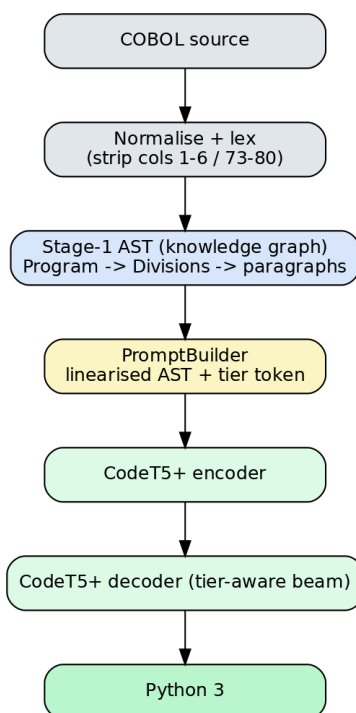


Figure 4. From COBOL source to the Stage-1 AST (a typed knowledge graph) to the CodeT5+ prompt and Python output.

3.5 Design Challenges and Solutions

- Data scarcity. No parallel corpus exists, so we manufactured one by execution-verified distillation (§4).
- Tokenizer incompatibility. CodeT5+’s tokenizer serializes special tokens in a format newer transformers reject; the trainer auto-detects this and materializes a patched tokenizer.
- Adapter loading: because the loader cannot apply LoRA adapters directly and the output folder also held a stale full model, evaluating the raw adapter folder silently used the old weights; we added a merge step that fuses the adapter onto the base and writes a clean checkpoint.
- Resource contention: running the data builder and trainer simultaneously starved the GPU and stretched each training step to nearly five minutes; serializing them cut training to roughly twenty minutes.

4. Training-Data Engineering

The proposal assumed training data would be available; it is not. Following the MultiPL-T recipe (Cassano et al. 2024), we generate targets with a teacher LLM and keep only translations whose Python output matches the COBOL output under the GnuCOBOL compiler—execution is the filter, not surface similarity, echoing TransCoder’s finding that computational accuracy, not BLEU, is the right signal (Rozière et al. 2020). Figure 6 shows the resulting funnel: of 588 candidate programs, only a small number are self-contained enough to run standalone, so the execution-verified “gold” yield is in the low tens.

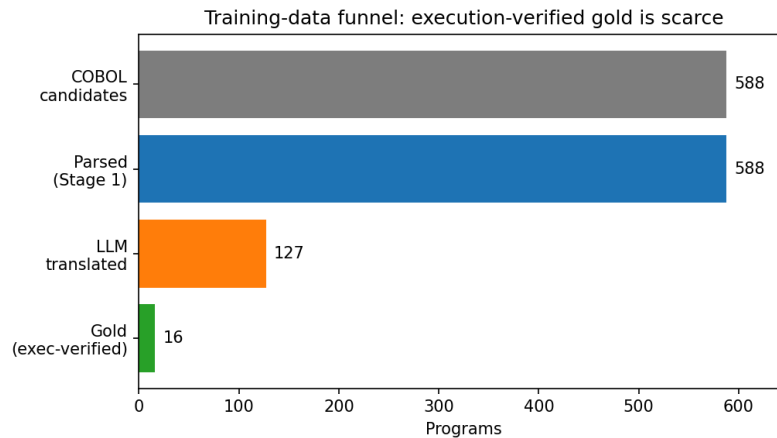


Figure 5. Training-data funnel. Execution-verified gold pairs are scarce because most NIST programs are conformance harnesses that do not run standalone.

To recover volume, programs that fail the execution gate are kept as “silver” pairs (teacher output, unverified) and used only in early curriculum tiers; gold pairs carry the High tier and all reporting. Two cheap quality filters proved important: dropping silver targets that do not even compile as Python (about 40 of 221), and up-weighting the scarce gold examples so they are not drowned by silver. The CodeXGLUE subset available locally is Java-to-C#, not Java-to-Python as the proposal stated, so it cannot warm-start a Python decoder; this is corrected in the data plan, with a Python warm-start identified as future work.

5. Incorporating Large Language Models

Large language models appear in three distinct roles, and it is worth separating their benefits from their costs. As a teacher, an LLM (a local qwen2.5-coder model, or a frontier GPT-4-class model) generates candidate

Python that the execution filter then verifies—this is the only practical way to manufacture supervision. As a refiner, the Stage-3 model improves readability and naming. As a judge, an LLM scores correctness and idiom where execution cannot run. The benefits are substantial: LLMs supply both the training targets and the evaluation oracle that the domain otherwise lacks. The challenges were equally real. Local models are slow—one early build spent 84 minutes to yield seven verified pairs—and produce malformed Python often enough that a compile filter is necessary. Judge choice matters: the same systems scored about 0.15 lower under a local 7-billion-parameter judge than under GPT-4o-mini, so absolute scores are only comparable within a fixed judge. These observations directly shaped the engineering: progress logging, a runnability pre-gate, incremental checkpointing of results, and judge-model discipline in reporting.

6. Experiments and Results

We evaluate with the proposal’s metrics: an LLM-as-judge weighted correctness score (the only oracle that runs on the full conformance corpus), execution-match on the self-contained subset, and CodeBLEU (Ren et al. 2020) where available. The neural model was trained for twenty epochs; its validation loss fell from 1.61 to 1.52 and plateaued (Figure 6 [loss]).

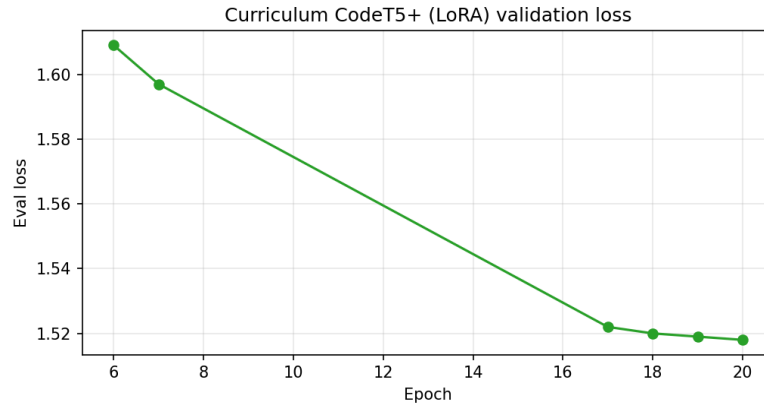


Figure 6. Curriculum CodeT5+ (LoRA) validation loss across training, showing convergence.

Figure 7 reports the full-NIST comparison under a GPT-4o-mini judge. The rule-based draft reaches 0.69 weighted correctness; the neural model reaches 0.54. Neither clears the 0.70 reporting gate, and the rule engine leads on every metric.

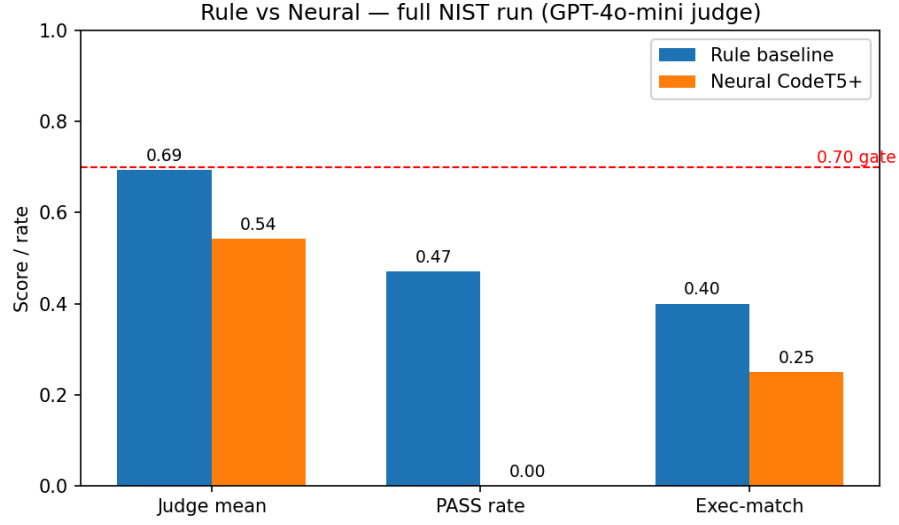


Figure 7. Rule vs. neural on the full NIST run (GPT-4o-mini judge): judge mean, PASS rate, and execution-match.

System	Judge	PASS	Exec-match	Files
Rule baseline	0.693	47%	40%	~348
Neural CodeT5+	0.542	0%	25%	460

Table 3. Full-NIST results (GPT-4o-mini judge).

To control program selection, we re-ran both systems on the same 50 programs under a local judge (Figure 8 and Table 4). The local judge scores everything lower—the rule baseline falls to 0.52—but the ordering is preserved: rule 0.52 versus neural 0.21. The drop is an artifact of the harsher judge, not a regression of the rule engine.

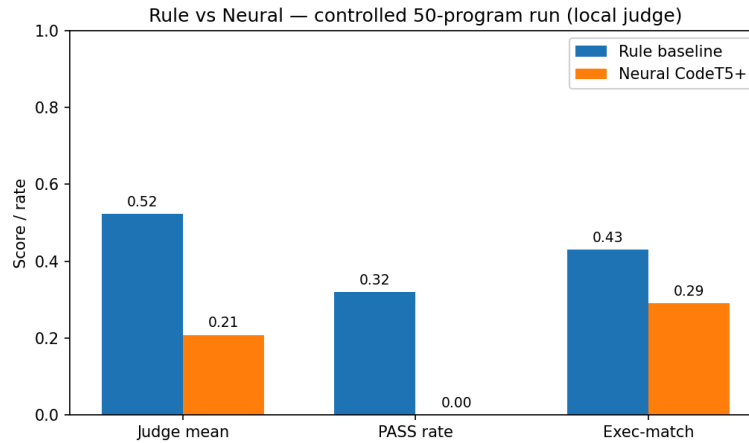


Figure 8. Controlled 50-program comparison under a local judge. Absolute scores drop for both systems; the rule-over-neural ordering is preserved.

System	Judge	PASS	Exec-match	Files
Rule baseline	0.523	32%	3/7	50
Neural CodeT5+	0.208	0%	2/7	50

Table 4. Controlled 50-program results (local qwen judge).

Interpretation. The neural model underperforms a hand-tuned rule engine, which is the expected outcome of fine-tuning a 220M model on roughly 200 examples—about 16 of them gold and the rest unverified silver—with no Python warm-start. The result is nonetheless meaningful: the curriculum CodeT5+ deliverable now exists, is trained end-to-end, merged, and evaluated under the proposal’s own oracles, where in the previous milestone it was untrained. The gap is a data-volume problem with a clear remedy, not an architectural dead end.

7. Discussion: Distance from the Proposal and Feedback

Against the proposal, the rule-based baseline, the curriculum CodeT5+ method, the NLI semantic validator, the LLM ensemble, and the identifier dictionary are all implemented; the LLM-judge and execution-accuracy metrics are reported; CodeBLEU exists but was not enabled in these runs; and the SWE-bench pass@1 metric and the human study remain out of scope. Against the advisor feedback, ANTLR4 was removed, the scope was concentrated on NIST/IBM with a curriculum-trained CodeT5+, and Chicago author-date citations are used here; applying CMOS to the proposal document itself remains the one open item. The principal limitation is training-data volume: execution-verified gold is scarce, silver is noisy, and no Python warm-start has yet been run.

8. Conclusion and Future Work

PARIVARTANA is a complete, evaluated, three-stage COBOL-to-Python pipeline whose neural component is now trained rather than hypothetical. The rule-based draft remains the strongest single stage, and the neural model’s gap is driven by data scarcity. Three levers close it, in priority order: a Python decoder warm-start before the COBOL curriculum; more gold via forward synthesis (translating tested Python into COBOL, verifying under GnuCOBOL, then inverting the pair) and the COBOLEval benchmark (Zorse Project 2024); and gold-weighted, fewer-epoch training to avoid overfitting silver. With those in place, the curriculum CodeT5+ model has a credible path to meet and exceed the deterministic baseline.

References

- Bengio, Yoshua, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. “Curriculum Learning.” In Proceedings of the 26th International Conference on Machine Learning, 41–48. ACM.
- Cassano, Federico, John Gouwar, Francesca Lucchetti, Claire Schlesinger, et al. 2024. “Knowledge Transfer from High-Resource to Low-Resource Programming Languages for Code LLMs.” Proceedings of the ACM on Programming Languages 8, OOPSLA.
- Challa, Kowshik Reddy, Sonith M V, Chiranjeevi B S, and Sridhar Chimalakonda. 2025. “COB2PY: A Non-AI, Rule-Based COBOL to Python Translator.” In Proceedings of ICSME 2025.
- Dau, Anh T. V., Hieu Trung Dao, Anh Tuan Nguyen, et al. 2024. “XMainframe: A Large Language Model for Mainframe Modernization.” arXiv:2408.04660.
- Dau, Anh T. V., Hieu Trung Dao, Anh Tuan Nguyen, et al. 2026. “COBOL-Coder: Domain-Adapted Large Language Models for COBOL Code Generation and Translation.” arXiv:2604.03986.
- Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. “LoRA: Low-Rank Adaptation of Large Language Models.” arXiv:2106.09685.
- Jimenez, Carlos E., John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In ICLR 2024.
- Kühne, Peter-Michael, et al. 2025. “Cross-Lingual Transfer in Programming Languages: An Extensive Empirical Study.” arXiv:2310.16937v3.
- Lu, Shuai, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, et al. 2021. “CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation.” In NeurIPS Datasets and Benchmarks Track.
- Micro Focus. 2022. COBOL Is Everywhere: Anecdotes Aside, Here Are the Facts. White paper. Newbury, UK: Micro Focus International.
- Platanios, Emmanouil Antonios, Otilia Stretcu, Graham Neubig, Barnábás Póczos, and Tom M. Mitchell. 2019. “Competence-Based Curriculum Learning for Neural Machine Translation.” In NAACL-HLT 2019, 1162–1172.

- Ren, Shuo, Daya Guo, Shuai Lu, Long Zhou, et al. 2020. “CodeBLEU: A Method for Automatic Evaluation of Code Synthesis.” arXiv:2009.10297.
- Rozière, Baptiste, Marie-Anne Lachaux, Lowik Chatussot, and Guillaume Lample. 2020. “Unsupervised Translation of Programming Languages.” In NeurIPS 33, 20601–20611.
- Thorne, James, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. “FEVER: A Large-Scale Dataset for Fact Extraction and VERification.” In NAACL-HLT 2018, 809–819.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, et al. 2017. “Attention Is All You Need.” In NeurIPS 30.
- Wang, Yue, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. “CodeT5+: Open Code Large Language Models for Code Understanding and Generation.” In EMNLP 2023, 1069–1088.
- Zorse Project. 2024. COBOLEval: Evaluating LLM-Generated COBOL. GitHub repository, BloopAI.

Appendix A: Reproduction

- Build data: `python scripts/build_training_data.py --datasets ibm_open_cobol nist_cobol --targets llm --provider ollama --execution-filter --keep-unverified --require-python-valid --gold-weight 5 --sample 50 --max-cobol-lines 300 --out artifacts/training_data/cobol_mix`
- Train: `python -m src.pipeline.stage2_neural.training --config configs/models.yaml --train artifacts/training_data/cobol_mix/train.jsonl --val artifacts/training_data/cobol_mix/val.jsonl --peft lora --epochs 10.`
- Merge: `python scripts/merge_lora.py --adapter artifacts/checkpoints/codet5p_cobol --out artifacts/checkpoints/codet5p_cobol_merged.`
- Evaluate and report: `python scripts/batch_run.py --dataset nist_cobol --max 50 --stage2 {rule,neural} ...;`
`python scripts/build_report.py.`
- Figures: `python scripts/visualize_model.py --which all --checkpoint artifacts/checkpoints/codet5p_cobol_merged.`

Appendix B: Key Hyperparameters

- Backbone: Salesforce/codet5p-220m (encoder-decoder, d_model 768, 12+12 blocks, ~223M params).
- LoRA: rank 16, alpha 32, dropout 0.05, targets q/k/v/o/wi/wo.
- Optimizer: learning rate 5e-5, weight decay 0.01, warmup ratio 0.06, batch size 16 with gradient accumulation 4, up to 20 epochs with early stopping (patience 3).
- Device profile: Apple M3 Pro (MPS), gradient checkpointing on.
- Execution filter: GnuCOBOL with whitespace-tolerant stdout match, 10–30s timeout.